

Edge AI Deployment

Samenvatting – MLOps4ECM

Doel van deze handleiding

Deze handleiding focust op het **deployen van AI-modellen op edge devices**: apparaten die lokaal inference uitvoeren, dicht bij de databron (sensoren, camera's, machines).

De nadruk ligt op:

- lage latency
- beperkte hardware resources
- energie-efficiëntie
- privacy en betrouwbaarheid
- hardwarebewuste optimalisatie

Edge AI vereist andere keuzes dan cloud- of server-side AI.

Introductie tot Edge AI

Wat is Edge AI?

Edge AI is **machine learning inference op fysieke apparaten**, buiten klassieke datacenters:

- dicht bij sensoren (camera, microfoon, accelerometer...)
- beperkte CPU, geheugen en opslag
- vaak batterijgevoed
- netwerk kan onbetrouwbaar of afwezig zijn

Modellen worden meestal **getraind in de cloud**, daarna **geoptimaliseerd en gedeployed op het device**.

Waar wordt Edge AI gebruikt?

- Industriële automatisatie (defectdetectie, onderhoud)
- Automotive en mobiliteit
- Medische en wearables
- Remote infrastructuur (energie, telecom, offshore)

Gemeenschappelijke nood: snelle, autonome beslissingen op lokale data.

Waarom Edge AI?

Latency

- Cloud inference: ~50–100 ms (best case)

- Edge inference: microseconden
- Cruciaal voor robots, drones en real-time systemen

Bandbreedte

- Sensors genereren enorme datastromen
- Uploaden is duur, traag of onmogelijk
- Edge AI verzendt enkel **samenvattingen of events**

Privacy & Compliance

- Gevoelige data blijft lokaal (GDPR, HIPAA)
- Minder risico op datalekken
- Vaak een wettelijke vereiste

Betrouwbaarheid

- Edge blijft werken zonder netwerk
- Geen single point of failure
- Vaak gecombineerd met cloud fallback

Kosten & Energie

- Cloud inference = doorlopende kosten
- Edge = vaste hardwarekost
- Lager energieverbruik door minder datatransmissie

Wanneer is Edge AI **geen** goede keuze?

- Zeer grote modellen (LLMs, grote transformers)
- Vaak wisselende modellen
- Samenwerkende inference over meerdere nodes

Gebruik Edge AI alleen als **latency, privacy, bandbreedte of autonomie** dat echt vereisen.

Edge hardware & accelerators

Waarom hardwarekeuze cruciaal is

In edge-projecten is hardware vaak:

- jarenlang in productie (10+ jaar)
- moeilijk of duur te vervangen
- onderhevig aan certificatie (CE, FCC, UL)

Foute keuzes leiden tot:

- herontwerpen
- supply-chain problemen

- onbetrouwbare systemen
-

Industriële randvoorwaarden

- Lange beschikbaarheid
- Robuustheid (temperatuur, trillingen, stof)
- Gecertificeerde modules
- Sterk software-ecosysteem

Goedkope hardware met slechte SDK's = verborgen kosten.

CPU, GPU en NPU

- **CPU**
 - flexibel, algemeen gebruik
 - best voor controlelogica en lichte ML
- **GPU**
 - sterk in parallelle berekeningen
 - geschikt voor vision en grotere modellen
- **NPU**
 - gespecialiseerde AI-accelerator
 - zeer efficiënt (INT8/FP16)
 - vaak ingebouwd in SoCs

Let op: hoge "TOPS"-waarden garanderen geen echte performance.

Hardware per workload

Inputtype	Typische hardware
Time series	MCU, TinyML
Audio	MCU / entry-level NPU
Beeld	NPU / embedded GPU
Video	Jetson / krachtige NPU
Transformers	zelden geschikt voor edge

Overzicht van belangrijke platforms

- **Microcontrollers (TinyML)** – STM32, Cortex-M
- **Embedded Linux SoCs** – NXP, TI, Rockchip
- **SBC's & modules** – Toradex, SolidRun, Raspberry Pi CM

- **NVIDIA Jetson** – krachtige embedded AI
 - **Externe accelerators** – Hailo
 - **FPGAs** – deterministische real-time AI
 - **Intel x86 & IPC's** – flexibel, hogere power
 - **PLC's met AI** – industriële controle + inference
-

Waarom modellen optimaliseren voor de edge?

Probleem

Modellen worden getraind op:

- krachtige GPU's
- veel RAM
- geen strikte latency-eisen

Edge devices hebben:

- beperkte RAM
- gedeelde GPU-geheugens
- strakke tijdslimieten

Zonder optimalisatie zijn veel modellen **onbruikbaar op edge**.

Inference-geheugen $\neq$ modelgrootte

- FP32: 4 bytes per parameter
- Activaties verbruiken vaak **5–10× meer geheugen**
- Vision-modellen zijn bijzonder zwaar

Voorbeeld:

- ResNet-50 → honderden MB RAM tijdens inference
-

Optimalisatietechnieken

Kwantisatie

Verlaag numerieke precisie:

- FP32 → FP16
- FP32 → INT8

Voordelen:

- 4× kleinere modellen
- 2–4× snellere inference
- minder geheugenverbruik

PTQ (Post-Training Quantization)

- Geen retraining nodig
- Snel en eenvoudig
- Kleine accuraatheidsdaling mogelijk

QAT (Quantization-Aware Training)

- Simuleert quantisatie tijdens training
 - Hogere accuraatheid
 - Meer complex
-

Pruning

Verwijdert overbodige delen van een netwerk:

- **Unstructured pruning**
 - verwijdert losse weights
 - weinig snelheidswinst
- **Structured pruning**
 - verwijdert filters / kanalen
 - echte speedups mogelijk

2:4 sparsity

- 2 van elke 4 weights = 0
 - Ondersteund door moderne NVIDIA GPU's
 - Reële performancewinst
-

Knowledge Distillation

- Groot model = *teacher*
 - Klein model = *student*
 - Student leert ook outputverdeling
 - Betere prestaties voor compacte modellen
-

Graph-level optimalisaties

Automatisch toegepast door runtimes:

- operator fusion
- constant folding
- memory reuse
- layout-optimalisatie

Onzichtbaar, maar zeer krachtig.

Efficiënte modelarchitecturen

Vision

- MobileNet V2/V3
- EfficientNet-Lite
- YOLO-Tiny
- MobileNet-SSD
- DeepLabv3+ (MobileNet)

Tijdreeksen & audio

- CNN-1D
- TCNs
- kleine LSTM/GRU
- spectrogram-CNNs

Gebruik bestaande efficiënte modellen i.p.v. zelf zoeken (NAS).

ONNX: van training naar deployment

Waarom ONNX?

ONNX = **framework-onafhankelijk modelbestand**:

- graph
- weights
- input/output specs

Export eenmaal, run overal.

ONNX vs ONNX Runtime

- **ONNX** = bestandsformaat
 - **ONNX Runtime** = inference-engine
-

Export uit PyTorch

1. Model laden en `eval()` zetten
2. Dummy input definiëren
3. `torch.onnx.export()`

Gebruik Netron om het model te inspecteren.

ONNX Runtime voor edge inference

Kenmerken

- Lichtgewicht
- Cross-platform
- Ondersteunt meerdere execution providers

Execution providers

- CPU
- CUDA
- TensorRT
- OpenVINO
- XNNPACK (ARM)

Optimalisaties

- automatische graph-optimalisaties
- dynamische en statische INT8-quantisatie
- FP16 ondersteuning voor GPU's

High-performance deployment met TensorRT

Wat is TensorRT?

- NVIDIA inference-engine
- Compileert ONNX naar hardware-specifieke binaries
- Maximaal gebruik van GPU/NPU-capaciteit

Werking

1. Graph parsing
2. Graph-optimalisatie
3. Kernel benchmarking
4. Memory planning
5. Engine serialisatie

Lange buildtijd, maar extreem snelle inference.

Precisies

- FP32: nauwkeurig, traag
- FP16: sneller, veilig
- INT8: snelst, vereist quantisatie
- FP8 / INT4: experimenteel

Model Optimizer (NVIDIA)

Wat doet Model Optimizer?

Optimaliseert **vóór export** in PyTorch:

- QAT
- structured pruning
- 2:4 sparsity
- NAS-achtige search

Pipeline

PyTorch

→ Model Optimizer

→ ONNX

→ TensorRT

→ Edge GPU

Conclusie

Edge AI vraagt:

- hardwarebewuste keuzes
- agressieve optimalisatie
- efficiënte modelarchitecturen
- gespecialiseerde runtimes

Deployment is waar machine learning écht begint.

Edge AI maakt AI:

- sneller
- energiezuiniger
- privacyvriendelijk
- robuust in de echte wereld